Clarity (visit2value) - Backend Technical Dossier & Feature Specification

Structured Evaluation Reference for Gemini Argon | Team AiX Global (Domain 2, SDG 8)

1. Executive Summary & System Overview

* Application Name: Clarity (internal package name: visit2value)
* Purpose: A multi-tenant, evidence-backed review-to-resolution operational workspace for small-to-medium businesses (demonstrated across a multi-property hotel 'harbour-house', a multi-branch restaurant 'nusa-table', and a salon 'luma-salon').
* Core Architectural Philosophy (Evidence First, Truthful Action): Raw customer reviews are persisted immutably before any AI processing occurs. Every AI-extracted observation must be anchored to a verified exact substring quote from the source text. Public replies cannot be drafted until an assigned operational task has been reported by staff and verified by an owner, and replies are programmatically guarded against claiming all issues are fixed when secondary complaints remain unresolved.

Technology Stack

* Runtime & Framework: Next.js 16.0.1 (App Router, nodejs runtime), React 19.2.0, TypeScript 5.9.3
* Database & ORM: PostgreSQL 16+ with Drizzle ORM (0.45.4) and pg (8.16.3) connection pooling
* Validation: Zod (4.1.12) for runtime request validation and structured LLM output parsing
* AI Providers: Jev / TypeSafe API (jev-latest via https://api.typesafe.ai/v1/systemone) + OpenAI Responses API (gpt-5-mini default, via openai SDK 6.8.0)
* Testing: Vitest (5.0.5) covering unit, domain, fixture, screening fallback, evidence span, rejection workflow, and HTTP route integration tests

2. API Layer & HTTP Contract

All backend endpoints execute exclusively on the server (runtime = 'nodejs'), keeping database credentials and AI API keys isolated from the browser.

Endpoints

* GET /api/state (src/app/api/state/route.ts): Accepts ?businessId=... (defaults to 'harbour-house'). Executes parallel scoped queries via getState() and returns the complete workspace snapshot: branches, employees, reviews, runs, observations, themes, memberships, tasks, taskReportAttempts (with attachment URLs), handoffs, replies, events, and themeSupport (distinct source review counts).
* POST /api/action (src/app/api/action/route.ts): Unified command dispatcher for 21 domain mutations. Runs assertActionScope(businessId, body) BEFORE executing any business logic to prevent cross-tenant (cross-business) ID tampering. Returns { ok: true, result } or HTTP 400 { ok: false, error, code } via publicActionError().
* GET /api/task-attachments/[id] (src/app/api/task-attachments/[id]/route.ts): Decodes base64 image evidence stored in task_report_attachments and streams binary image bytes with strict security headers (X-Content-Type-Options: nosniff, Cache-Control: private, no-store).
* GET /api/prepared-reviews (src/app/api/prepared-reviews/route.ts): Stub endpoint returning [] to ensure no external scraping or unverified feed ingestion.

Supported Mutation Actions (POST /api/action)

* Ingestion & Analysis: intake, analyze, analyze_batch (up to 5 pending reviews per call), manual_classify
* Evidence & Triage Management: membership (move or remove observation-to-theme link), theme_status (owner_confirmed, dismissed, suggested), severity (1, 2, or 3)
* Task Lifecycle: propose_task, edit_task, decide_task (approve, reject, defer), start_task, report_task (with image attachments), verify_task / resolve_task, reject_task_report
* Reply Lifecycle: draft_reply, prepare_replies (concurrency-bounded batch of 6), edit_reply, approve_reply, reject_reply, approve_reply_batch (1 to 300 safe drafts), simulate_publish

3. Database Schema & Persistence Architecture

Defined in src/data/schema.ts and managed through 5 additive, idempotent SQL migrations (0001_initial.sql through 0005_task_report_attachments_and_analysis_timings.sql):

* 1. Multi-Tenant Hierarchy: businesses -> branches (properties/locations) -> employees (branch-scoped staff).
* 2. Immutable Review Evidence (reviews): Stores originalText, rating (1-5), reviewedAt, language (en | ms | mixed | unknown), isSynthetic, isHistoricalImport, processingStatus (pending, processing, complete, generic, needs_review, failed), and processingStage. Enforces unique constraint on (business_id, source, source_review_id).
* 3. AI Provenance & Telemetry (analysis_runs & observations): analysis_runs tracks every attempt per review (decision, screeningProvider, screeningModel, extractionProvider, extractionModel, matchingProvider, matchingModel, promptVersion, mode, safeError, and stageTimings JSONB). observations stores individual extracted points (category, aspect, kind, sentiment, summary, normalizedDescription, exact evidenceQuote, startOffset, endOffset, and model provenance).
* 4. Issue Grouping (themes & theme_memberships): themes are branch-scoped clusters (operational_issue, suggestion, positive_pattern) with status (suggested, owner_confirmed, dismissed, recurrence), severity (1-3), computed priority (urgent, high, medium, low, insight), and priorityReason. theme_memberships links an observation to at most one theme (uniqueIndex on observation_id), tracking whether the link is an ai_suggestion, fixture, or owner_correction.
* 5. Operational Workflows (tasks, task_report_attempts, task_report_attachments, handoffs): tasks tracks title, description, assignee, due date, status, reportVersion, approvalSnapshot (JSONB), verification metadata, rejection metadata, and recurrenceOfTaskId. task_report_attempts stores an immutable versioned log of staff completion reports (version, note, decision, decisionReason). task_report_attachments stores up to 5 validated base64 image attachments per report attempt. handoffs stores simulated external task handoff payloads (simulated_queued).
* 6. Public Replies (replies & reply_draft_versions): replies is 1-to-1 with reviews (unique on review_id), linked to a verified task_id and theme_id. Tracks draftVersion, approvedVersion, partialWarning, partialApproved, and publishedAt. reply_draft_versions maintains an immutable version history of every AI-generated and owner-edited reply draft.
* 7. Audit Log (activity_events): Append-only, business-scoped event ledger recording actor, role, entity type/ID, description, and structured metadata.

4. Core Backend Features & Domain Invariants

Feature 1: Idempotent Intake & Source-Evidence Conflict Detection

* Implemented in intake() (src/domain/service.ts) and intakeSchema (src/domain/analysis.ts).
* Validates branch ownership against the business, integer rating (1-5), ISO timestamp with timezone offset, text length (5-3000 chars), and isSynthetic: true.

* Idempotency vs. Tamper Detection: If a review with the same (business_id, source, source_review_id) arrives with identical text, branch, rating, and timestamp, it returns { reviewId, duplicate: true }. If any of those fields differ under the same source ID, it throws a Feed Conflict error and preserves the original evidence untouched.

Feature 2: Dual-Mode AI Pipeline (live vs. fixture) with Provider Fallback

* Controlled by AI_MODE ('live' or 'fixture') in analyzeReview() (src/domain/service.ts).
* Stage 1 - Screening (screenWithFallback in src/domain/screening.ts): In live mode, calls Jev (screenWithJev) with an 8-second timeout and up to 2 attempts. Jev returns a 'noul' probability score [0, 1]. Scores >= 0.6 are true (contains specific feedback), <= 0.4 are false (generic). Scores between 0.4 and 0.6 are treated as uncertain, automatically triggering an OpenAI Fallback (screenWithOpenAI). Network or HTTP failures on Jev also fall back seamlessly to OpenAI while recording the Jev failure in safeError.
* Stage 2 - Span-Grounded Extraction & Anti-Hallucination Guard (extractWithOpenAI & verifyEvidence): Instead of letting the LLM free-generate quotes (which risks paraphrasing or hallucination), evidenceSpans() segments the original review into numbered sentences (s1, s2, ...) using Intl.Segmenter. OpenAI is constrained via a dynamic z.enum(ids) schema to select an exact evidenceSpanId for each observation. verifyEvidence() then verifies that every quote is a literal character-for-character substring of originalText (normalizing only straight/curly quotes) and slices the exact substring from the original review.
* Stage 3 - Context-Aware Theme Matching (candidateThemes & matchObservationsWithOpenAI): Filters existing themes to the same branch and compatible feedback type (operational_complaint -> operational_issue, suggestion -> suggestion, praise -> positive_pattern). Ranks candidates by lexical overlap and boosts themes that have active tasks. Short-circuits the LLM call if an exact multi-word title match exists or no candidates overlap. Validates that OpenAI only returns candidate IDs from the supplied list (preventing cross-branch or cross-business injection). Unmatched observations in live mode automatically create new suggested themes.
* Per-Stage Performance Telemetry: Tracks jevScreeningMs, openAIFallbackScreeningMs, screeningMs, extractionMs, issueMatchingMs, databasePersistMs, and pipelineMs in analysis_runs.stage_timings.

Feature 3: Distinct-Review Support Counting & Triage Priority Engine

* Implemented in distinctReviewCount() (src/domain/themes.ts), recalculatePriority() (src/domain/service.ts), and priorityFor() (src/config/priority.ts).
* Anti-Inflation Rule: If a single guest mentions an issue 3 times in one review, or multiple observations from the same review map to the same theme, it counts as 1 distinct source review: COUNT(DISTINCT o.review_id).
* Priority Matrix: Combines owner-adjustable severity (1-3), total distinct reviews, recent reviews (within 180 days), and post-resolution recurrence:
* - urgent: Severity 3 (e.g., safety, accessibility, or pests - even with 1 review).
* - high: Recurrence detected, OR (severity >= 2 and >= 5 distinct reviews with recent activity), OR >= 10 distinct reviews with recent activity.
* - medium: >= 2 distinct reviews OR severity >= 2.
* - low: All other operational issues.
* - insight: Non-operational themes (suggestion and positive_pattern).

Feature 4: Operational Task Lifecycle, Optimistic Concurrency & Image Evidence

* Implemented in proposeTask(), decideTask(), reportTask(), verifyTask(), and rejectTaskReport() (src/domain/service.ts).

* Advisory Locking: proposeTask() acquires a PostgreSQL transaction-level advisory lock (pg_advisory_xact_lock(hashtext(themeId))) to prevent race conditions from creating duplicate active tasks for the same issue.
* Simulated Orbyte Handoff: Task approval requires a same-branch employee and valid due date, snapshots all supporting { reviewId, quote } pairs into approvalSnapshot, and writes a simulated payload via localHandoff (src/integrations/orbyte/handoff.ts).
* Staff Work Reports & Binary Image Validation: Assigned staff submit a completion report (>= 10 chars) and up to 5 image attachments (image/jpeg, image/png, image/webp, max 2 MB each, max 6 MB total, validated for strict base64 encoding and sanitized filenames). Each submission increments report_version and creates an immutable task_report_attempts record.
* Version-Checked Verification & Rejection: Owners must pass the exact reportVersion they are reviewing. Conditional SQL WHERE clauses (eq(tasks.reportVersion, reportVersion)) block stale approvals/rejections if an employee submits a newer report concurrently. Rejecting a report requires a 10-500 character reason and transitions the task to needs_rework.
* Automatic Recurrence Detection: If a new review supporting a theme arrives after a task was resolved (review.createdAt > task.resolvedAt), refreshRecurrenceStatus() transitions the theme to recurrence, elevates priority to high, logs a possible_recurrence event, and unlocks proposing a new follow-up task linked via recurrenceOfTaskId.

Feature 5: Truthful Reply Drafting & Partial-Resolution Safeguards

* Implemented in createReply(), unresolvedReviewComplaints(), assertReplyApproval() (src/domain/replies.ts), and simulatePublish().
* Prerequisites: A reply can only be drafted if the linked task is resolved with a non-empty resolutionNote and the review has an observation belonging to that task's theme.
* Cross-Complaint Inspection: Before drafting, unresolvedReviewComplaints() inspects all other operational_complaint observations in the same review. Any complaint that is ungrouped or belongs to a theme without a resolved task is flagged as unresolved.
* Truthfulness Guardrails:
* 1. Unresolved complaints are passed into the OpenAI reply prompt so the draft explicitly acknowledges what was fixed while noting other items remain under review.
* 2. A conspicuous partialWarning is stored on the reply record.
* 3. Regex Universal-Claim Blocker (impliesAllConcernsResolved): If a review has unresolved complaints, assertReplyApproval() requires explicit partialApproved: true AND scans the reply text to reject overbroad claims (e.g., 'all issues have been resolved', 'everything is fixed', 'no concerns left').
* 4. Safe Batch Approval (approveReplyBatch): Batch approval (1-300 replies) is strictly restricted to drafts with no partialWarning and theme severity < 3. High-severity or partial replies must be reviewed and approved individually.
* 5. Version-Locked Publishing: Editing an approved reply increments draft_version, records a row in reply_draft_versions, and revokes approved_ready status back to draft. simulate_publish verifies status === 'approved_ready' and approvedVersion === draftVersion, logging a reply_publish_denied audit event if an unapproved draft is submitted for publishing.

5. Security, Resilience & Honest Demo Boundaries

* Tenant Isolation: Every GET /api/state and POST /api/action validates businessId and checks entity ownership across reviews, observations, themes, tasks, and replies via assertActionScope. Demo Boundary: No session authentication; businessId, role (owner/staff), and employeeId are client-supplied selector values for judge walkthroughs.

* Prompt Injection Defense: Both Jev and OpenAI system prompts explicitly instruct the model to treat review and observation text as untrusted data, and all outputs are enforced via strict Zod schemas (zodTextFormat).
* Error Normalization: normalizeOpenAIError() and publicActionError() map raw DB/SDK exceptions into safe, user-readable messages without leaking API keys, stack traces, or SQL queries.
* External Integrations: Real server-side adapters for Jev and OpenAI with timeouts, retries, and fallbacks (Jev -> OpenAI, OpenAI reply -> template_fallback). Demo Boundary: Review ingestion, Orbyte task handoff (localHandoff), and reply publishing (simulated_published) are local simulations with zero external side effects.
* Scalability & Concurrency: Advisory locks on task proposals; optimistic version locking (reportVersion, draftVersion) on task reports and reply edits/approvals; bounded concurrency (6) on batch reply preparation. Demo Boundary: GET /api/state returns the full business snapshot for client-side filtering/pagination, and analyze runs synchronously inside the HTTP request rather than a background worker queue.